

CS1CA Computer Systems Architecture

Information Representation – 3

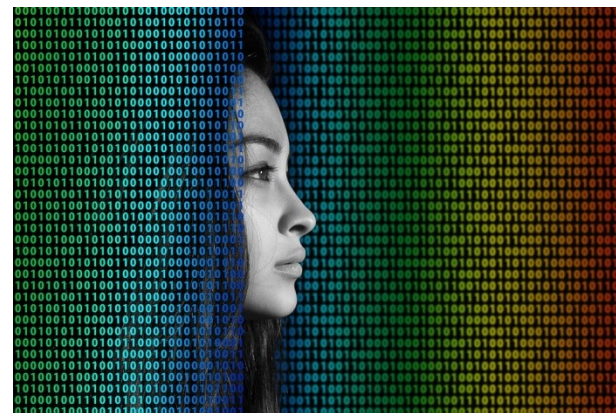
Week 1 lecture 4 of autumn semester 2025

Lectured by

Dr. Hong Wei

h.wei@reading.ac.uk

Room 133 in Polly Vacher Building



Outline of the topic

- Introduction: what is information?
- Decimal, binary and hexadecimal number systems
 - Conversion between number systems
- Internal representation of characters
- Internal representation of integers
- Arithmetic and logic operations
- Internal representation of real numbers
- IEEE 754 floating point format

In this lecture

- The following contents are delivered.
 - Internal representation of integers
- The learning outcomes are listed below.
 - To be able to use a fixed length of bit patterns to represent integers
 - To be able to use Two's complement to represent negative integers

Internal representation of integers – 1

- Numbers are originally input as a sequence (string) of ASCII characters;
- A conversion from ASCII string to internal binary representation must be done; reverse conversion on output to monitor and printer;
- **Fixed length of bit patterns** is used for integer representation. It depends on **architecture of machines**, such as 32-bit or 64-bit systems.
- It also depends on **data types** in programming language and databases.

Internal representation of integers – 2

- The fixed length n bits determine the number of integers which are representable.
- For example, 2^n
when $n = 8$, $2^8 = 256$ integers representable;
when $n = 16$, $2^{16} = 65,536$ integers representable, and so on.
- **Data types** include **signed and unsigned** integers.
- Positive and negative integers are sharing the overall numbers representable in the equal manner.
- For example, when $n = 8$, 256 integers can be represented by the combination of the 8 bits. Among them, 128 are used for positive integers and 128 are used for negative integers.
- The starting positive is 0, and the starting negative is -1.

Internal representation of integers – 3

For example: when $n = 3$ bits, there are $2^3 = 8$ integers which can be represented.

- For unsigned integer, the 8 binary integers are 000, 001, 010, 011, 100, 101, 110, and 111, corresponding to decimal numbers of 0, 1, 2, 3, 4, 5, 6, and 7.
- For signed integers, the range is of -4, -3, -2, -1, 0, 1, 2, 3, *i.e.* from -4 to 3.
- We can confidently convert decimal 0, 1, 2, and 3 to binary 000, 001, 010, and 011, but **how can we convert a negative number to binary?**
- See **Two's complement** in the next slide.

Two's complement: a method to deal with negative numbers

- Given any positive number N_b , *the two's complement* $2^n - N_b$ used for its negative, where n is number of bits.

Example: find binary representations of 5 and -5 by using 8-bit patterns

$$N = 5_{10} = 05_{16} = 00000101_2$$



Two's complement

- A simple method**: Given a fixed length of a binary number as a positive (or negative) number, flip all bits of the binary number and add 1 to the flipped number. This will provide you the corresponding negative (or positive) number.

Range of fixed length integers

- The range of integers represented in n bits are

$$-2^{n-1} \leq N_{ext} \leq 2^{n-1} - 1$$

n	2^n	Max. positive $2^{n-1}-1$	Min. negative -2^{n-1}
8	256	127	-128
16	65,536	32,767	-32,768
32	4,294,967,296	2,147,483,647	-2,147,483,648

The sign bit of signed integers

- The most significant bit (MSB) is **ALWAYS** the sign bit irrespective of number of bits used for a signed integer.

Example: if $n = 16$, then

(a) The negative integer -1 is represented by FFFF_{16} , and $-32,768$ is 8000_{16}

(b) The positive integer 1 is represented by 0001_{16} , and $32,767$ is $7FFF_{16}$

- A positive number has its MSB as 0, whilst a negative number has its MSB as 1.

Examples: signed integers

- Represent the given decimal to the fixed length of 8-bit notations.
 - $N = -124 = 1\text{xxxxxxxx}$,
 - apply Two's complement, we have $256 - 124 = 132$
 - Convert 132 to binary, we have 10000100.
 - So that $\langle -124 \rangle_{10} = \langle 10000100 \rangle_2$
 - $N = 82 = 0\text{xxxxxxxx}$
 - We convert the decimal to binary straight away.
 - $\langle 82 \rangle_{10} = \langle 01010010 \rangle_2$
 - $N = -172$ (problem?)

Problem of fixed length encoding of numbers

- If an integer can't be represented by the given number of bits, what would happen? For example, how to use 3 bits to represent a decimal number of 9?
- It is an **overflow issue**.
- We will discuss the overflow issue associated with binary arithmetic and logic operations in the next week.

Arithmetic and logic operations

Arithmetic operations will be used in the discussion
of overflow.

The knowledge of Boolean algebra will be used in
binary logic operations.

Mathematical background – Boolean algebra which will be used in binary logic operations

- **Boolean algebra**: basic principles of logical reasoning by using 1 (TRUE) and 0 (FALSE).
- Operations of Boolean algebra

NOT	
A	$\sim A$
0	1
1	0

AND			
$\&$	0	1	B
0	0	0	
1	0	1	
A			

OR			
$ $	0	1	B
0	0	1	
1	1	1	
A			

XOR: EXCLUSIVE-OR			
$\wedge$	0	1	B
0	0	1	
1	1	0	
A			

- **Boolean operations** correspond to logic operations, or digital logic, which is the foundation of digital electronic computers.

Examples: logic operations with respect to Boolean algebra

- Two binary values

$$A = [0110]_2 = 6; \quad B = [1100]_2 = -4$$

- NOT operation: $\sim$

$$\sim A = [1001]_2 \quad \sim B = [0011]_2$$

- Logic operations: $\&$ AND; $|$ OR; $\wedge$ XOR

– Find the results of $A \& B$, $A | B$, and $A \wedge B$:

0110	0110	0110
$\&$ 1100	$ $ 1100	$\wedge$ 1100
0100	1110	1010

Laws of Boolean operations

- Commutative law
 - ✓ $A \& B = B \& A$
 - ✓ $A | B = B | A$
- AND law
 - ✓ $A \& 0 = 0; \quad A \& 1 = A;$
 - ✓ $A \& A = A; \quad A \& (\sim A) = 0$
- OR law
 - ✓ $A | 0 = A; \quad A | 1 = 1;$
 - ✓ $A | A = A; \quad A | (\sim A) = 1$

End of the lecture

Self-assessment Exercises

- Convert the following positive numbers to 8-bit binary notation:

(a) 39

(b) 52

(c) 108

(d) 122

- Convert the following negative numbers to binary:

(a) -39

(b) -52

(c) -108

(d) -122

Use 8-bit two's complement notation for the negative numbers.